STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: TEST_Q004_ODDEVEN
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Testing Debugging Concepts using AI	STATUS: In Progress

Practice Worksheet – Q004 – Find Odd or Even Number Parity Testing

Design comprehensive parity evaluation test suites: bitwise vs modulo parity checks, negative integer edge cases, extreme integer boundaries, and JUnit 5 parameterized test execution.

KEY LEARNING OBJECTIVES:

- Evaluate algorithm efficiency: Modulo operator (`n % 2`) vs Bitwise AND (`(n & 1) == 0`).
- Identify and fix the classic Java negative modulo parity bug (`-5 % 2 == -1`).
- Assert parity across zero, negative integers, and Integer.MAX_VALUE / Integer.MIN_VALUE boundaries.
- Implement data-driven parameterized tests utilizing JUnit 5 @CsvSource.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze the mathematical definition of parity across the entire integer number line.
Step 2: Implement the parity function using both modulo and bitwise techniques.
Step 3: Formulate test cases covering positive numbers, negative numbers, zero, and extreme limits.
Step 4: Execute parameterized assertions to verify that all inputs map to correct boolean values.

EXERCISE 4.1: ALGORITHM COMPARISON & THE NEGATIVE MODULO BUG

Problem Statement: In Java, `n % 2 == 1` returns `false` for negative odd numbers (e.g., `-5 % 2` evaluates to `-1`). Explain why this bug occurs in naive implementations and contrast the two robust solutions: `n % 2 != 0` vs bitwise `(n & 1) != 0` in terms of CPU cycle efficiency.

Reference Baseline / Scenario / Code:

```
// Naive Bug:
public boolean isOdd(int n) { return n % 2 == 1; } // Fails for -5!
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Negative Modulo Behavior Analysis: _____

[2] Bitwise Parity Formula: _____

[3] Performance & CPU Cycle Comparison: _____

EXERCISE 4.2: COMPREHENSIVE PARITY TEST MATRIX SPECIFICATION

Problem Statement: Draft a test matrix for `boolean isEven(int n)`. Include 6 distinct categories: (1) Standard positive even (2, 4, 100), (2) Standard positive odd (1, 3, 99), (3) Zero (0), (4) Negative even (-2, -4, -100), (5) Negative odd (-1, -3, -99), and (6) Boundaries (`Integer.MAX_VALUE`, `Integer.MIN_VALUE`).

Reference Baseline / Scenario / Code:
Input: -2147483648 (Integer.MIN_VALUE) | Expected: true (Even) | Category: Extreme Limit

STUDENT SOLUTION & CODE WORKSPACE:
[1] Parity Test Classification Table: _____
[2] Sample Integer Values: _____
[3] Expected Boolean Outputs: _____

EXERCISE 4.3: JUNIT 5 @PARAMETERIZEDTEST WITH @CSVSOURCE IMPLEMENTATION

Problem Statement: Implement a data-driven unit test `testNumberParity(int input, boolean expectedIsEven)` using `@ParameterizedTest` and `@CsvSource` containing all 6 categories from Exercise 4.2. Assert that the bitwise implementation satisfies all test cases without failure.

Reference Baseline / Scenario / Code:
```
@ParameterizedTest(name = "Run {index}: isEven({0}) should be {1}")
@CsvSource({"2, true", "3, false", "0, true", "-4, true", "-7, false", "2147483647, false"})
```

STUDENT SOLUTION & CODE WORKSPACE:
[1] JUnit 5 Parameterized Annotations: _____
[2] CsvSource Data Entries: _____
[3] Assertion Logic Method: _____

EXERCISE 4.4: AI PROMPT FOR AUTOMATIC BENCHMARK & MUTATION TESTING

Problem Statement: Construct a prompt for an AI assistant to generate a JMH (Java Microbenchmark Harness) benchmark and Pitest mutation testing suite comparing modulo vs bitwise parity checking over an array of 10,000,000 random integers.

Reference Baseline / Scenario / Code:

```
Act as a Java Performance Engineer. Generate a JMH benchmark comparing modulo vs bitwise parity checking...
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] AI Benchmark Prompt Persona: _____

[2] JMH Benchmark Directives: _____

[3] Mutation Testing Validation Criteria: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Negative Parity:** Explicitly handles negative odd integers without modulo truncation bugs.
- [ ] **Bitwise Accuracy:** Bitwise (n & 1) syntax correctly parenthesized and verified.
- [ ] **Boundary Limits:** Integer.MIN_VALUE and Integer.MAX_VALUE verified.
- [ ] **Parameterized Testing:** CsvSource covers clean distribution of odd, even, and negative numbers.
- [ ] **Benchmark Evaluation:** JMH benchmark methodology clearly defined.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is `n % 2 != 0` safer in Java than `n % 2 == 1` when determining odd parity?*
- *How does mutation testing help verify that your test suite truly catches subtle conditional operator bugs?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases